Ali Raza Chaudhry Abdul Hadi

COMSATS University Islamabad, Lahore Campus
B.S. Computer Engineering
Supervisor: Dr. Taimoor Naeem

Final Year Project Defense — 2025

Outline

1. Introduction
2. System Architecture
3. Input & Preprocessing
4. AI Models
5. Edge Deployment
6. Access Control
7. Results
8. Conclusion

Problem Statement

Challenge

Outdoor vehicle gates must identify **license plates** and verify **vehicle identity** under fog, rain, glare, and night — yet many systems rely on **plate-only** decisions vulnerable to misreads and cloned plates.

Limitations of existing systems:

- Poor performance in adverse weather
- Plate-only authorization (spoofing risk)
- Cloud-dependent (latency, privacy)
- Fragmented LPR + VMMR pipelines

Our solution:

- Weather-aware preprocessing
- Dual YOLOv8n LPR pipeline
- EfficientNet-B4 VMMR (245 classes)
- Dual-modality gate policy
- Edge AI on Orange Pi 5 Pro

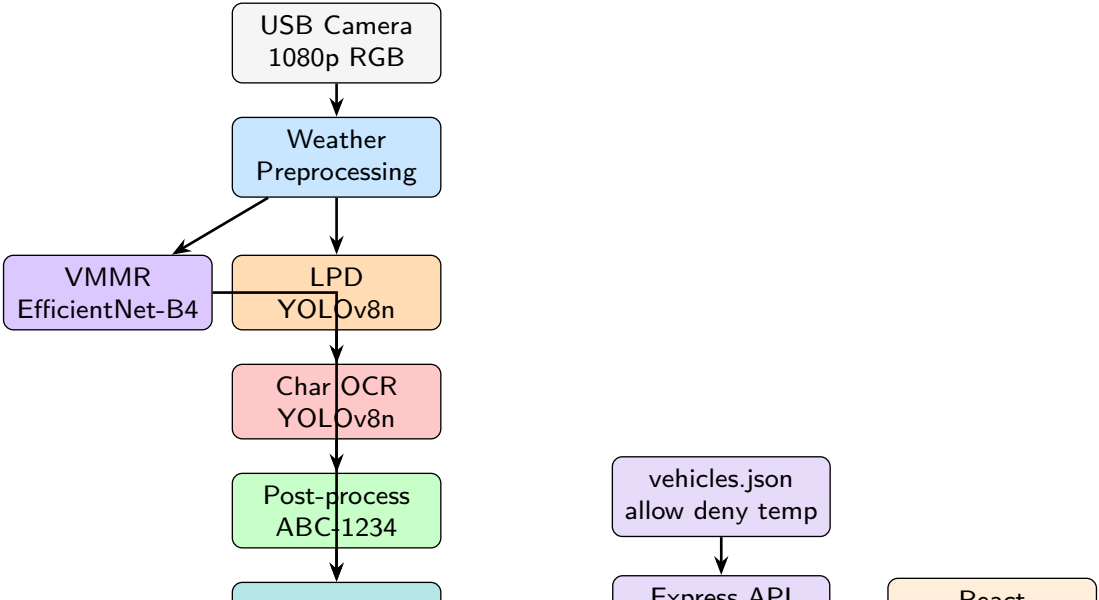
Objectives & Contributions

1. Train **YOLOv8n** for license plate detection (LPD)
2. Train **YOLOv8n** for character/digit detection (38 classes)
3. Train **EfficientNet-B4** for 245 make–model families (VMMR)
4. Integrate fog, rain, and night preprocessing before inference
5. Implement dual-modality access control (plate + make/model)
6. Deploy on **Orange Pi** with web dashboard for monitoring

Novelty

Coupled pipeline: weather restoration → dual YOLO LPR → VMMR → JSON → allow/deny/alert policy — reproducible end-to-end.

High-Level System Architecture



End-to-End Data Flow

1. **Capture:** USB camera acquires full vehicle frame (RGB)
2. **Preprocess:** Apply fog / rain / night restoration (or pass-through)
3. **Stage 0 — VMMR:** Classify make–model on full preprocessed image
4. **Stage 1 — LPD:** Detect plate bounding box (conf ≥ 0.25)
5. **Stage 2 — OCR:** Detect characters on cropped plate ($+4$ px padding)
6. **Post-process:** Sort left$\rightarrow$right, format as `LETTERS-DIGITS`
7. **Deploy:** Orange Pi builds JSON, HTTP POST to web server
8. **Policy:** Match plate + make/model against database $\rightarrow$ gate action

Edge performance (Orange Pi 5 Pro CPU)

Total latency: **700–1000 ms/image** ($\sim$1 FPS) — acceptable for single-lane gate access.

Input Image Handling

Sources:

- USB 1080p camera at gate lane (live)
- Batch image upload (JPG, PNG, BMP, WEBP, TIF)

Processing:

- Convert to RGB: `PIL.Image.convert('RGB')`
- Unsupported formats re-saved as JPEG
- Original resolution preserved; models resize internally

Hardware:

- Orange Pi 5 Pro (16 GB RAM)
- 5V 5A USB-C power supply
- Optional GPIO relay for physical gate

Preprocessing Modes

Flag	Filter
`none`	Pass-through
`auto`	Auto-detect weather
`fog`	DCP defog
`rain`	Vertical median
`night`	CLAHE + gamma

Fog / Haze Removal — Dark Channel Prior

Function: `defog_image()` — runs *before* all AI models

1. Compute dark channel (13×13 morphological erosion)
2. Estimate atmospheric light (top 0.1% darkest pixels)
3. Transmission map + guided filter (r=50, ε=10^{-3})
4. Scene radiance recovery: $(I - A)/t + A$
5. Residual haze removal on bright low-variance regions
6. Gray-world color balance
7. Adaptive gamma correction ($\gamma \in [0.6, 1.8]$)
8. CLAHE on LAB L-channel (clip=2.5, 8×8 tiles)
9. HSV saturation boost ($\times1.2$)
10. Bilateral filter + unsharp mask

Why classical CV?

- No extra DL model to train
- Fast on Orange Pi CPU
- Interpretable parameters
- Works without paired training data

Key idea

Dark pixels in outdoor haze reveal depth; dehazing restores plate edges and vehicle texture.

Rain & Night Preprocessing

Rain Removal — `derain_image()`

- Edge-protection mask (bilateral + Canny)
- Vertical median filters: 15, 11, 7 px kernels
- Brightening guard (prevents halos on glyphs)
- Rain spot detection (area 2–250 px)
- Telea inpainting (r=3)

Night Enhancement — `preprocess_night()`

- Black-level lift on LAB L-channel (up to +55)
- Adaptive CLAHE (clip 4.5–8.5, 6×6)
- Adaptive gamma ($\gamma \in [0.28, 0.62]$)
- Global gain if mean < 100 ($\times$1.0–2.5)
- HSV saturation $\times$1.55
- Unsharp mask for edge recovery

All three branches feed the same downstream YOLO + EfficientNet weights.

Model Summary

Stage	Model	Classes	Input	Epochs	Weight file	mAP / Acc.
LPD	YOLOv8n	1 (plate)	384	50	`plate_best.pt`	mAP@0.5 = 98.1%
Char OCR	YOLOv8n	38	640	100	`char_best.pt`	mAP@0.5 = 98.1%
VMMR	EfficientNet-B4	245	380	50	`efficientnet_b4_best.pth`	Top-1 = 96.2%

LPD Dataset

Roboflow *License Plate Project* v1

OCR Dataset

Roboflow *My First Project* v4
(Pakistani plates)

VMMR Dataset

Custom 245-class folder dataset
(17,254 test images)

Stage 1 — License Plate Detection (YOLOv8n)

Architecture: YOLOv8 nano (Ultralytics)

- Single detection class: `plate`
- Pretrained on COCO, fine-tuned on Roboflow v1
- Training: 50 epochs, batch=32, imgsz=384
- Inference: conf ≥ 0.25, NMS IoU=0.7

Output: Bounding box (x_1, y_1, x_2, y_2)

```
plate_results = plate_model.predict(
    source=img_np, conf=0.25)
```

Metrics (Epoch 50)

Precision	**98.50%**
Recall	**95.53%**
mAP@0.5	**98.07%**
mAP@0.5:0.95	70.73%

Why YOLOv8n?

Small (∼6 MB), fast on Pi CPU (80–200 ms), sufficient accuracy for gate LPD.

Stage 2 — Character / Digit Detection (YOLOv8n)

Task: Detect individual characters on cropped plate

- 38 classes: digits 0–9, letters A–Z, `YY`, a
- Training: 100 epochs, batch=16, imgsz=640
- Each character = one detection box (not seq2seq OCR)

Advantages over traditional OCR:

- Handles skewed / dirty plates
- No separate OCR engine dependency
- End-to-end trainable with plate data

Metrics (Epoch 100)	
Precision	**98.99%**
Recall	**96.88%**
mAP@0.5	**98.07%**
mAP@0.5:0.95	76.11%

Character assembly:

1. Get x-center of each detection
2. Sort left → right
3. Split letters vs digits
4. Format: `ABC-1234`

Post-Processing — OCR Assembly

Function: `fix_license_plate_format(detections)`

1. Classify each detection as letter (A–Z) or digit (0–9)
2. Sort by horizontal x-center (reading order)
3. Take up to **3 letters** + last **4 digits**
4. Return Pakistan-style format: `LETTERS-DIGITS`

Example:

`[(A,42), (B,68), (C,95), (1,120), (2,145), (3,170), (4,195)]` → **ABC-1234**

Visual annotation: Yellow bar = VMMR label; lime box = plate + text overlay.

Output States

Result	Meaning
`ABC-1234`	Valid plate
`NO_TEXT`	No valid chars
`NO_CHARS`	Plate found, no OCR
`NO_PLATE`	LPD failed

Stage 0 — VMMR (EfficientNet-B4)

Architecture:

- EfficientNet-B4 backbone (ImageNet pretrained)
- Classifier: Dropout(0.4) → Linear(1792 → 245)
- 245 make–model classes (e.g. `Honda_Civic`)

Training:

- Input: 224×224 RandomResizedCrop + augmentations
- Optimizer: AdamW, lr=10^{-4}, label smoothing 0.1
- Scheduler: ReduceLROnPlateau; early stop patience 5
- 50 epochs on Tesla T4 GPU

Test Metrics

Top-1 Accuracy	**96.24%**
Top-5 Accuracy	**99.39%**
Macro Precision	96.34%
Macro Recall	96.12%
Macro F1	96.18%

Dual-modality role

VMMR output is cross-checked against registered make/model at the gate — catches cloned plates.

Orange Pi 5 Pro Deployment

Hardware:

- Orange Pi 5 Pro, 16 GB RAM
- USB 1080p camera, 5V 5A PSU
- Wi-Fi for JSON transmission
- Optional GPIO → gate relay

Deployment scripts:

- `setup_orangepi.sh` — environment setup
- `download_models.py` — ∼207 MB weights
- `test_models.py` — verify all 3 models
- `run_webcam.py -preprocess auto`
- `run_images.py <folder>`

Model Files on Pi

File	Size	Role
`plate_best.pt`	∼6 MB	LPD
`char_best.pt`	∼6 MB	OCR
`efficientnet_b4_best.pth`	∼75 MB	VMM
`classes.txt`	∼5 KB	Labe

Latency (CPU)

LPD: 80–200 ms OCR: 30–100 ms
VMMR: 200–800 ms
Total: 700–1000 ms — fully offline, no cloud required.

JSON Payload & Web API

Orange Pi → Express.js server (port 3001)

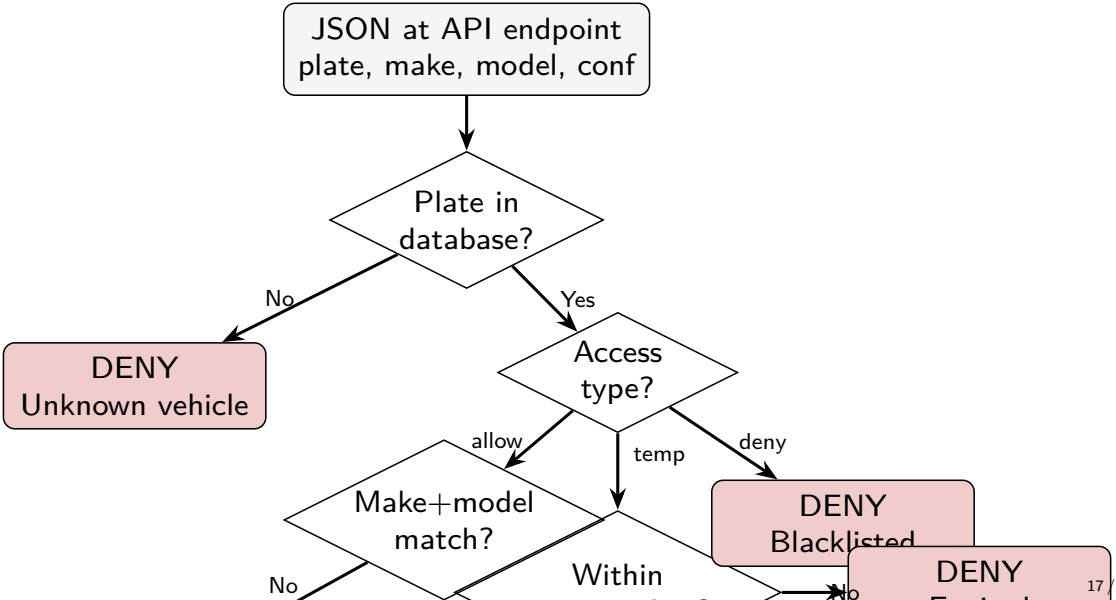
```
POST /api/predict
{
  "plate": "ABC-1234",
  "make": "Honda",
  "model": "Civic",
  "confidence": 0.94,
  "weather": "Fog",
  "timestamp": "2025-06-23T10:30:00Z",
  "status": "Allowed"
}
```

Key API endpoints:

- `POST /api/predict` — inference result
- `POST /api/events` — log gate event
- `GET /api/events` — paginated history
- `CRUD /api/vehicles/*` — manage rules
- `POST /api/override` — manual gate control
- `POST /api/auth/login` — JWT auth

Dashboard: React + LiveFeed (10 s polling)

Gate Decision Logic — Dual-Modality Policy



Vehicle Database & Access Types

Storage: `vehicles.json` (permanent rules)

```
{
  "licensePlate": "ABC-1234",
  "make": "Toyota",
  "model": "Corolla",
  "accessType": "allow",
  "ownerName": "John Doe"
}
```

Access Types

Type	Behavior
`allow`	Permanent whitelist → GRANT (after dual check)
`deny`	Permanent blacklist → DENY immediately
`temp`	Allowed only between `entryTime` and `expiry`

Roles:

- **Admin:** add allow / deny / temp rules
- **User:** add temp visitors only
- Manual override via dashboard

Quantitative Results

Model	Precision	Recall	mAP@0.5	Notes
YOLOv8n LPD	98.50%	95.53%	98.07%	Epoch 50, imgsz 384
YOLOv8n Char	98.99%	96.88%	98.07%	Epoch 100, imgsz 640
EfficientNet-B4 VMMR	—	—	—	Top-1: 96.24%, Top-5: 99.39%

LPD Strength

High precision (98.5%) — few false plate detections at gate.

OCR Strength

38-class char detector handles Pakistani plate formats.

VMMR Strength

245-class fine-grained recognition with 96%+ Top-1 on 17K test images.

Qualitative Results

Demonstrated capabilities:

- Fog/haze images: visible improvement after DCP defog before LPD
- Rain scenes: vertical streak suppression preserves plate characters
- Night frames: CLAHE + gamma restores plate readability
- Multi-plate parking scenes: LPD detects multiple boxes
- Annotated output: vehicle label (yellow) + plate box (lime) + text

Integrated pipeline

Single notebook chains preprocessing $\rightarrow$ VMMR $\rightarrow$ LPD $\rightarrow$ OCR $\rightarrow$ formatted plate string with side-by-side before/after grids.

Limitations & Future Work

Current limitations:

- $\sim$1 FPS on Orange Pi CPU (not real-time video)
- VMMR trained at 224^2, inferred at 380^2
- Dashboard simulates some predictions for demo
- GPIO gate relay not fully wired in repo
- Snow/smog not in scope

Future work:

- ONNX / TensorRT export for faster Pi inference
- Full Pi $\rightarrow$ API live integration
- GPIO relay for physical gate motor
- HTTPS + API key authentication
- Supabase production database migration
- Expand weather branches (snow, smog)

Conclusion

Summary

We built an end-to-end **all-weather smart vehicle gate** system that combines:

1. Classical weather restoration (fog, rain, night)
2. Two-stage YOLOv8n license plate reading
3. EfficientNet-B4 vehicle make–model recognition (245 classes)
4. Dual-modality access control (plate + vehicle identity)
5. Edge deployment on Orange Pi with web dashboard

Thank You

Questions?